HƯỚNG DẪN THỰC HÀNH / 01

## Hướng dẫn cải tiến thuật toán MBA

# MBA-A: căn chỉnh theo mức hỗ trợ dữ liệu và tăng dần trọng số KL

Phiên bản hướng dẫn 1.0 | Ngày 29/10/2026

**Mục tiêu.** Xây dựng một biến thể có thể kiểm chứng của Multi-Behavior Alignment (MBA), giữ nguyên cơ chế khử nhiễu và thay đổi cách truyền thông tin từ hành vi nhấp, mua vào mô hình sở thích tiềm ẩn.

**Trạng thái.** Đây là tài liệu thiết kế và hướng dẫn sửa mã. Các đoạn mã chưa được tích hợp vào dự án; chưa có kết quả huấn luyện MBA-A. Không xem các giả thuyết trong tài liệu là kết quả thực nghiệm.

### Điểm xuất phát từ bài báo

| Bộ dữ liệu | Recall@20 | NDCG@20 |
|---|---|---|
| Beibei | 0,1742 | 0,1046 |
| Taobao | 0,0812 | 0,0435 |
| MBD | 0,5677 | 0,3285 |

Nguồn: Bảng 2 của bài báo [1], MBA dùng MF. Đây là mốc công bố, không phải kết quả chạy lại với giao thức validation mới.

### Cơ sở từ dữ liệu trong dự án

| Thống kê trên tập mua train | Beibei | Taobao |
|---|---|---|
| Số người dùng | 21.716 | 48.658 |
| Trung vị số sản phẩm đã mua/người | 10 | 4 |
| Người có không quá 5 sản phẩm đã mua | 2.092 (9,63%) | 40.672 (83,59%) |

Các thống kê được đếm theo sản phẩm khác nhau trong các file buy_train.csv hiện có [3]. Sau khi tách validation, phải tính lại trọng số chỉ trên phần train còn lại.

### Cách sử dụng tài liệu

Đọc công thức ở trang 2; chuẩn hóa giao thức và tham số ở trang 3-4; triển khai trọng số và loss ở trang 5-6; chạy thí nghiệm theo trang 7-8; báo cáo và đối chiếu theo trang 9.

HƯỚNG DẪN THỰC HÀNH / 02

# 1. Thiết kế thuật toán MBA-A

## 1.1. Giữ nền tảng của MBA

Ký hiệu P_f là phân phối từ mô hình mua huấn luyện trước; P_g từ mô hình nhấp; P_t từ mô hình sở thích tiềm ẩn. L_denoise là tổng các số hạng khử nhiễu và được giữ như triển khai MBA gốc.

$$L_{MBA} = L_{denoise} + \alpha [KL(P_f || P_t) + KL(P_g || P_t)]$$

MBA-A giữ hai teacher, mô hình nền MF, bốn mô hình điều kiện của phần khử nhiễu và quy trình cập nhật luân phiên. Tên MBA-A chỉ dùng để phân biệt biến thể đề xuất trong tài liệu này.

## 1.2. Trọng số theo mức hỗ trợ dữ liệu của người dùng

Với mỗi người dùng u và hành vi b thuộc {mua, nhấp}, gọi $n_u^b$ là số sản phẩm khác nhau đã tương tác trong train. Chọn $k_b$ là trung vị số tương tác dương của hành vi đó; nếu không có tương tác thì đặt $k_b = 1$.

$$r_u^b = n_u^b / (n_u^b + k_b)$$
$$D_u = 2\varepsilon + r_u^f + r_u^g$$
$$w_u^f = 2(\varepsilon + r_u^f) / D_u$$
$$w_u^g = 2(\varepsilon + r_u^g) / D_u$$

Khởi đầu với ε = 0,05. Ta có $w_u^f + w_u^g = 2$; khi hai mức hỗ trợ bằng nhau thì mỗi trọng số bằng 1. Dùng k riêng cho từng hành vi để lượng nhấp lớn không tự động lấn át lượng mua.

**Lưu ý:** Đây là trọng số theo lượng dữ liệu, không phải xác suất dữ liệu sạch. Người dùng có nhiều lượt nhấp vẫn có thể tạo ra tín hiệu nhiễu; hiệu quả phải được xác nhận qua thí nghiệm.

## 1.3. Tăng dần lực căn chỉnh

$$\alpha(e) = \alpha_{max} \times \min(1, (e + 1) / E_w)$$

e bắt đầu từ 0. $E_w$ là số epoch tăng dần trọng số, có thể thử 5, 10 hoặc 20. Nếu $E_w = 0$, dùng $\alpha(e) = \alpha_{max}$ ngay từ đầu. Ví dụ $\alpha_{max} = 10$ và $E_w = 10$ cho α lần lượt là 1, 2, ..., 10 rồi giữ nguyên.

## 1.4. Hàm mục tiêu và suy luận

$$L_{MBA-A} = L_{denoise} + \alpha(e) [w_u^f KL(P_f || P_t) + w_u^g KL(P_g || P_t)]$$

Trọng số áp dụng theo từng mẫu trước khi lấy trung bình batch. Khi suy luận, giữ $s_{ui} = \beta p_{ui}^t + (1 - \beta) p_{ui}^f$ như MBA gốc. Không thêm mạng mới và không cần trọng số w ở bước suy luận.

HƯỚNG DẪN THỰC HÀNH / 03

# 2. Chuẩn hóa dữ liệu và đánh giá

Thực hiện phần này cho cả MBA đối chứng và MBA-A. Thay đổi giao thức đánh giá không được tính là đóng góp cải tiến thuật toán.

## 2.1. Chia train / validation / test

**Bước 1.** Giữ nguyên file buy_test hiện có làm tập đánh giá cuối. Khử các cặp trùng trong từng tập và kiểm tra train mua không trùng test mua.

**Bước 2.** Tách khoảng 10% tương tác của mỗi người từ buy_train làm validation bằng một split_seed cố định. Giữ ít nhất một sản phẩm mua trong train; người chỉ có một tương tác không đóng góp mẫu validation. Chỉ đánh giá trên người có ground truth hợp lệ.

**Bước 3.** Tách validation nhấp từ pv_train theo nguyên tắc tương tự. Giữ nguyên ánh xạ ID và không gian user/item qua các tập; không suy luận lại kích thước embedding chỉ từ phần train đã giảm.

**Bước 4.** Chọn teacher mua bằng validation mua; chọn teacher nhấp bằng validation nhấp; chọn mô hình mục tiêu và β bằng validation mua. Chỉ mở kết quả test sau khi chốt cấu hình.

## 2.2. Giả định về hành vi nhấp phụ trợ

load_all() trong data_utils.py hiện gộp pv_train, pv_test và buy_train. Điều này cho phép sử dụng lượt nhấp từ file pv_test khi dự đoán mua. Đó là một giả định về dữ liệu phụ trợ có sẵn; không tự động đồng nghĩa dùng nhãn mua test.

Giao thức chính đề xuất: chỉ dùng phần nhấp train và các cặp mua train được phép gộp; không gộp pv_test hoặc mua validation/test vào train. Áp dụng một quy tắc cho tất cả B0-B3 và ghi rõ khi báo cáo.

**Lưu ý:** Các CSV hiện có không đủ để bảo đảm phép chia theo thời gian. Không gọi đây là đánh giá theo thời gian nếu chưa có timestamp và quy tắc cutoff tương ứng.

## 2.3. Những chỗ phải đổi luồng dữ liệu

| Vị trí | Công việc |
|---|---|
| data_utils.py | Trả thêm validation mua/nhấp; chỉ dựng ma trận từ dữ liệu train được phép. |
| main.py | Truyền validation vào các lời gọi pretrain() và train(); giữ test cho test_only. |
| train.py | Các lần đánh giá theo epoch phải nhận validation; lưu checkpoint tốt nhất theo validation. |
| LightGCN / cache .npz | Tạo lại ma trận kề từ split mới; không dùng cache cũ. |

Khi đánh giá mua, loại các sản phẩm đã mua trong train khỏi danh sách ứng viên; không loại mọi sản phẩm từng nhấp. Giữ tập ứng viên và quy tắc loại trừ giống nhau giữa các phương pháp.

HƯỚNG DẪN THỰC HÀNH / 04

# 3. Chuẩn bị mã và tham số

## 3.1. Lưu phiên bản đối chứng

Lưu mã nguồn gốc hoặc ghi lại commit trước khi sửa. Phân biệt: bản tác giả; bản đối chứng đã sửa lỗi triển khai và dùng validation; bản MBA-A. So sánh chính là MBA-A với bản đối chứng trong cùng giao thức.

## 3.2. Sửa lỗi ánh xạ lambda1 trong main.py

Trong dictionary truyền vào main(...), thay dòng đang lấy args.lambda0 cho lambda1 bằng:

```
'lambda1': args.lambda1,
```

Nếu không sửa, --lambda1 trên dòng lệnh không tác động đúng. Áp dụng cùng cách sửa cho đối chứng và cải tiến; không tính đây là ưu thế của MBA-A.

## 3.3. Khai báo tùy chọn trước parser.parse_args()

```
parser.add_argument(
    "--adaptive_alignment", action="store_true"
)
parser.add_argument(
    "--alignment_warmup", type=int, default=0
)
```

Bổ sung hai trường vào dictionary truyền đến main(...):

```
'adaptive_alignment': args.adaptive_alignment,
'alignment_warmup': args.alignment_warmup,
```

## 3.4. Kiểm soát cấu hình thử nghiệm đầu tiên

| Thành phần | Lựa chọn ban đầu |
|---|---|
| Mô hình / teacher / mô hình h | MF / MF / MF |
| Embedding / batch / learning rate | 32 / 2048 / 0,001 |
| Lấy mẫu âm NSR | 1 cho cả đối chứng và MBA-A |
| Suy luận | Giữ công thức phối hợp β của MBA |
| Seed chia dữ liệu | Cố định, tách biệt với seed khởi tạo mô hình |
| Checkpoint teacher | Dùng chung giữa B0-B3 cho từng seed |

Lưu riêng thư mục checkpoint và log của mỗi cấu hình. Tên checkpoint hiện tại không chứa đầy đủ α, lịch tăng dần và loại biến thể; dùng chung thư mục có thể ghi đè kết quả.

HƯỚNG DẪN THỰC HÀNH / 05

# 4. Tính trọng số theo người dùng

Thêm đoạn sau vào cuối TrainModel.__init__() trong train.py. Mã dùng numpy và torch đã được import trong file. Hai ma trận phải là dữ liệu train sau khi tách validation.

```
self.adaptive_alignment = param.get(
    "adaptive_alignment", False
)
self.alignment_warmup = param.get(
    "alignment_warmup", 0
)

if (self.adaptive_alignment or self.alignment_warmup > 0):
    if self.NSR != 1:
        raise ValueError("MBA-A first experiment requires NSR=1")

def support_score(mat):
    counts = np.asarray(
        mat.tocsr().getnnz(axis=1), dtype=np.float32
    )
    positive_counts = counts[counts > 0]
    k = (float(np.median(positive_counts))
         if len(positive_counts) else 1.0)
    return counts / (counts + k)

support_buy = support_score(self.train_mat_buy)
support_pv = support_score(self.train_mat_pv)
eps = 0.05
denominator = support_buy + support_pv + 2 * eps

self.alignment_weight_buy = torch.tensor(
    2 * (support_buy + eps) / denominator,
    dtype=torch.float32, device=self.device
)
self.alignment_weight_pv = torch.tensor(
    2 * (support_pv + eps) / denominator,
    dtype=torch.float32, device=self.device
)
```

## Ý nghĩa và phạm vi

getnnz(axis=1) đếm số sản phẩm khác nhau của mỗi người trên ma trận tương tác nhị phân. Trọng số được tính một lần và tra theo uid trong mỗi batch; không cập nhật bằng gradient.

Nếu ma trận chứa giá trị 0 được lưu tường minh, hãy loại các phần tử 0 trước khi đếm. Nếu thay đổi split, phải tính lại thống kê; tuyệt đối không lấy số tương tác từ validation hoặc test.

Giữ NSR=1 trong vòng thử đầu tiên. Khi mở rộng NSR, phải kiểm tra cách lặp uid và ghép tensor âm trong toàn bộ vòng train, không chỉ riêng hàm trọng số.

HƯỚNG DẪN THỰC HÀNH / 06

# 5. Thay các số hạng KL

Thêm phương thức sau vào lớp TrainModel. KL_loss đã được import từ loss_func.py. Hàm vẫn trả về vector theo mẫu; torch.mean() hiện có trong vòng train giữ nguyên.

```
def alignment_loss(self, teacher_logits, target_logits,
                   users, behavior, epoch):
    per_sample_kl = KL_loss(
        teacher_logits.detach(), target_logits
    )
    if self.adaptive_alignment:
        if behavior == "buy":
            weights = self.alignment_weight_buy[users]
        elif behavior == "pv":
            weights = self.alignment_weight_pv[users]
        else:
            raise ValueError("Unknown behavior")
        if weights.shape != per_sample_kl.shape:
            raise ValueError("Check uid and loss tensor shapes")
        per_sample_kl = weights * per_sample_kl

    ramp = 1.0
    if self.alignment_warmup > 0:
        ramp = min(1.0, (epoch + 1) / self.alignment_warmup)
    return self.alpha * ramp * per_sample_kl
```

## Ví dụ thay một vị trí

```
kl_pos_loss_pv = self.alignment_loss(
    pos_prediction_pv_pretrain, pos_prediction_target,
    uid, behavior="pv", epoch=epoch
)
```

Thay toàn bộ 8 vị trí self.alpha * KL_loss(...) trong TrainModel.train(), ở cả vòng mẫu mua và vòng mẫu chỉ nhấp. Chọn behavior theo teacher đang cung cấp logits:

| Tên số hạng KL | behavior | Số vị trí |
|---|---|---|
| kl_pos_loss_pv, kl_neg_loss_pv | pv | 4 (ở hai vòng) |
| kl_pos_loss_buy, kl_neg_loss_buy | buy | 2 |
| kl_neg_loss_rest, kl_neg_loss_both | buy | 2 |

Không thay KL trong pretraining hoặc thay số hạng khử nhiễu bằng trọng số này. Để chạy B0: tắt adaptive_alignment và đặt alignment_warmup = 0; công thức KL khi đó trở về đối chứng.

HƯỚNG DẪN THỰC HÀNH / 07

# 6. Lệnh chạy và quản lý checkpoint

Chỉ dùng các lệnh này sau khi tích hợp mã trang 3-6. Chạy trong thư mục dự án bằng PowerShell. Đường dẫn .venv giả định môi trường đã được cài các thư viện cần thiết.

## 6.1. Huấn luyện hai teacher một lần cho mỗi seed

```
New-Item -ItemType Directory -Force output/exp_b0

.\.venv\Scripts\python.exe main.py `
  --dataset beibei --train_method pre `
  --model MF --pretrain_model MF --h_model MF `
  --seed 2020 --lambda0 1e-4 `
  --pretrain_early_stop_rounds 20 `
  --folder ./output/exp_b0
```

## 6.2. MBA đối chứng B0

```
.\.venv\Scripts\python.exe main.py `
  --dataset beibei --train_method mba `
  --model MF --pretrain_model MF --h_model MF `
  --seed 2020 --alpha 10 --C_1 100 --C_2 100 `
  --beta 0.8 --lambda0 1e-4 --lambda1 1e-6 `
  --NSR 1 --alignment_warmup 0 `
  --early_stop_rounds 20 --folder ./output/exp_b0
```

## 6.3. Chuẩn bị thư mục MBA-A B3

```
New-Item -ItemType Directory -Force output/exp_b3
Copy-Item output/exp_b0/pretrain_*.pt output/exp_b3/
```

Chỉ sao chép checkpoint cùng bộ dữ liệu, seed, split và cấu hình teacher. Chạy lệnh B0 ở trên nhưng đổi --folder thành ./output/exp_b3, thêm --adaptive_alignment và đổi --alignment_warmup thành 10.

**Lưu ý:** Không chạy huấn luyện pre trong thư mục B3 nếu mục tiêu là dùng lại đúng teacher của B0. Nếu có GPU, mặc định chương trình dùng cuda; nếu không, thêm --device cpu vào mọi lệnh.

## 6.4. Đánh giá test cuối cùng

Sau khi chọn cấu hình bằng validation, chạy lại lệnh tương ứng với --test_only để đọc checkpoint đã chọn và đánh giá test. Giữ đúng dataset, model, seed, lambda0, lambda1, folder và β. Lưu log ra file riêng bằng chuyển hướng PowerShell nếu cần.

Các giá trị α = 10, C_1 = C_2 = 100 và β = 0,8 là điểm khởi đầu để thử, không phải tuyên bố cấu hình tối ưu cho mọi dữ liệu.

HƯỚNG DẪN THỰC HÀNH / 08

# 7. Ma trận thực nghiệm và kiểm tra

## 7.1. Thử riêng từng thành phần

| Mã | Trọng số người dùng | Tăng dần KL | Mục đích |
|---|---|---|---|
| B0 | Không | Không | MBA đối chứng |
| B1 | Có | Không | Đo tác dụng trọng số |
| B2 | Không | Có | Đo tác dụng lịch KL |
| B3 | Có | Có | MBA-A đầy đủ |

Giữ cùng split, tập ứng viên, quy tắc loại sản phẩm đã mua và cùng teacher cho từng seed. Dùng ngân sách tinh chỉnh tương đương để so sánh cấu hình cuối; không chỉ tinh chỉnh B3 rồi so với B0 mặc định.

## 7.2. Phạm vi tìm kiếm ban đầu

| Tham số | Giá trị thử |
|---|---|
| $\alpha_{max}$ | 1, 10, 100 |
| $E_w$ | 5, 10, 20 |
| Hệ số nhân trung vị để đặt $k_b$ | 0,5; 1; 2 |
| $\beta$ | 0,7; 0,8; 1 |

Trước tiên cố định k theo trung vị và thử B1, B2. Chỉ mở rộng tìm k nếu có tín hiệu hữu ích. Khi thử k, áp dụng hệ số nhân trước phép n/(n+k) và ghi lại cấu hình. Giữ $C_1$, $C_2$ và các tham số khử nhiễu như đối chứng trong vòng đầu.

## 7.3. Kiểm tra trước khi huấn luyện dài

1. Kiểm tra train, validation, test mua không trùng cặp. Không dùng validation/test trong thống kê trọng số hoặc đồ thị train.

2. Kiểm tra $w_{mua} + w_{nhấp}$ xấp xỉ 2, các trọng số dương, hữu hạn; trường hợp mức hỗ trợ bằng nhau cho hai trọng số bằng 1.

3. Tắt hai cải tiến phải khôi phục cách tính KL đối chứng. Kiểm tra gradient của teacher không được cập nhật qua loss căn chỉnh, còn target nhận gradient hữu hạn.

4. Chạy 1-2 epoch để kiểm tra kích thước uid/logits/loss, NaN và việc lưu, nạp checkpoint. Kiểm tra $\alpha$ tăng đúng và đạt $\alpha_{max}$ sau $E_w$ epoch.

5. Kiểm tra test_only sử dụng đúng checkpoint chọn theo validation, không phải epoch cuối. Giữ nguyên quy tắc tính Recall và NDCG.

## 7.4. Lặp lại và đo chi phí

Dùng ít nhất 3 seed; nếu có tài nguyên, thử 2020, 2021, 2022, 2023, 2024. Giữ split_seed cố định để tách biến động khởi tạo khỏi biến động chia dữ liệu. Báo cáo thời gian pretraining, huấn luyện mục tiêu, số epoch và bộ nhớ GPU nếu đo được.

HƯỚNG DẪN THỰC HÀNH / 09

# 8. Báo cáo, kết luận và tài liệu nguồn

## 8.1. Mẫu bảng kết quả cần điền

| Bộ dữ liệu | Bản | R@20: TB ± SD | N@20: TB ± SD |
|---|---|---|---|
| Beibei | B0 | Chưa chạy | Chưa chạy |
| Beibei | B1 | Chưa chạy | Chưa chạy |
| Beibei | B2 | Chưa chạy | Chưa chạy |
| Beibei | B3 | Chưa chạy | Chưa chạy |
| Taobao | B0 | Chưa chạy | Chưa chạy |
| Taobao | B3 | Chưa chạy | Chưa chạy |

Bảng chính thức cần có cả R@10 và N@10. Tính mức tăng tương đối: Δ = 100 × (điểm_B3 - điểm_B0) / điểm_B0. Phân biệt phần trăm tương đối với điểm phần trăm; không so trực tiếp với bảng bài báo nếu giao thức dữ liệu đã đổi.

## 8.2. Phép kiểm tra giả thuyết quan trọng

Chia người dùng theo số sản phẩm mua trong train: 1-5, 6-10, trên 10. Báo cáo điểm và số người trong từng nhóm. Nếu chỉ tăng ở nhóm nhiều dữ liệu, chưa có bằng chứng cải tiến giúp nhóm ít tương tác như giả thuyết ban đầu.

So sánh chênh lệch theo cùng seed. Nếu báo cáo ý nghĩa thống kê, nêu rõ phép kiểm định, đơn vị lấy mẫu và khoảng tin cậy; không coi các người dùng qua nhiều seed là các quan sát hoàn toàn độc lập. Tránh kết luận từ một lần chạy tốt nhất.

## 8.3. Cách diễn giải đóng góp

Đóng góp dự kiến: điều chỉnh lượng tri thức từ hai hành vi theo mức hỗ trợ dữ liệu của người dùng và theo giai đoạn huấn luyện. Chi phí thêm gồm hai trọng số/người và các phép nhân theo mẫu; kiến trúc mạng và công thức suy luận được giữ nguyên.

Chỉ kết luận MBA-A tốt hơn nếu kết quả lặp lại trên test, sau lựa chọn bằng validation, ủng hộ điều đó. Tài liệu không xác nhận tính mới học thuật; cần khảo sát nghiên cứu liên quan trước khi sử dụng làm tuyên bố đóng góp khoa học.

## Tài liệu nguồn và phạm vi kiểm chứng

[1] Xin Xin và cộng sự (2023). Improving Implicit Feedback-Based Recommendation through Multi-Behavior Alignment. SIGIR 2023. Công thức (12), Bảng 2-4. DOI: 10.1145/3539618.3591697. arxiv.org/abs/2305.05585. Bản đính kèm: docs/2305.05585v1.pdf.

[2] Mã nguồn tại thư mục dự án: main.py (tham số), train.py (huấn luyện và KL), loss_func.py (KL và khử nhiễu), data_utils.py (nạp dữ liệu), evaluate.py (đánh giá). Hướng dẫn dựa trên trạng thái mã đã đọc trong cuộc trao đổi; nếu mã thay đổi, đối chiếu lại trước khi áp dụng.

[3] Dữ liệu dùng cho thống kê trang 1: data/beibei/beibei_buy_train.csv và data/taobao/taobao_buy_train.csv. Các số liệu này mô tả file train gốc hiện có, không mô tả split mới.

Các đoạn Python trong trang 5-6 đã được kiểm tra cú pháp khi tạo tài liệu. Chưa kiểm thử tích hợp hoặc huấn luyện; các kiểm tra ở trang 8 vẫn cần được thực hiện sau khi áp dụng.